

Reviewer Pack — Minimal Deligne-Lusztig Parameters and Communication Complex...

Entry 0ada91edca54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 09:25:28 UTC

Minimal Deligne-Lusztig Parameters and Communication Complexity Rank

Entry ID: 0ada91edca54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 09:25:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Deligne-Lusztig Theory) **Field B** (complexity object): Communication Complexity

Statement:

For every n -communication protocol π , the minimal Deligne-Lusztig parameters ($dl\pi$) of its associated representation are linearly correlated with its communication complexity rank (r_π), such that $dl\pi = \Theta(r_\pi)$.

Rationale (proposer's reasoning):

Deligne-Lusztig theory studies the action of the symmetric group on polynomial functions, which can be seen as a generalization of linear representations. The minimal Deligne-Lusztig parameters may capture intrinsic complexity in representation theory and thus could provide insights into communication complexity, where understanding fundamental complexity is key.

Taxonomy category: DELIGNE_LUSZTIG (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): afc4ca73c00c6b7c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between minimal Deligne-Lusztig parameters ($dl\pi$) and communication complexity rank (r_π) for 30 random seeds exceeds 0.7, with no seed producing a correlation below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Deligne-Lusztig Theory" AND "communication complexity" - "minimal Deligne-Lusztig parameters" AND communication complexity - "rank of communication complexity" AND Deligne-Lusztig

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_protocol(n):
        vertices = list(range(n))
        edges = []
        for v in vertices:
            for u in range(v + 1, n):
                if random.choice([True, False]):
                    edges.append((v, u))
        return vertices, edges

    def deligne_lusztig_parameters(G, V):
        vertices, edges = G
        n = len(vertices)
        dl_param = 0
        for v in vertices:
            neighbors = [u for u in vertices if (v, u) in edges or (u, v) in edges]
            dl_param += len(neighbors)
        return Fraction(dl_param, n * (n - 1))

    def communication_complexity_rank(G):
        vertices, edges = G
        rank = 0
        for v in vertices:
            neighbors = [u for u in vertices if (v, u) in edges or (u, v) in edges]
            rank += len(neighbors)
        return rank
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    vertices, edges = generate_protocol(n)
    dl_param = deligne_lusztig_parameters((vertices, edges), vertices)
    r_pi = communication_complexity_rank((vertices, edges))
    results.append((dl_param, r_pi))

if len(results) < 30:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

dl_params = [r[0] for r in results]
ranks = [r[1] for r in results]

mean_dl_param = sum(dl_params) / len(dl_params)
mean_rank = sum(ranks) / len(ranks)

covariance = sum((dl_params[i] - mean_dl_param) * (ranks[i] - mean_rank) for i in range(len(dl_params)))
variance_dl_param = sum((dl_params[i] - mean_dl_param) ** 2 for i in range(len(dl_params))) / len(dl_params)
variance_rank = sum((ranks[i] - mean_rank) ** 2 for i in range(len(ranks))) / len(ranks)

pearson_corr_coeff = covariance / (math.sqrt(variance_dl_param) * math.sqrt(variance_rank))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_corr_coeff,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": pearson_corr_coeff > 0.7 and all(r >= 0.5 for r in ranks),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")

```

```

elif any(not r["conjecture_holds"] for r in results) and any(r["metric_value"] < 0.5 for r in
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed={first_
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'conjecture_holds': False, 'counterexample': 'not_enough_instances'}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson correlation coefficient', 'metric_value': None, 'instances_tested': 0}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b67d964.py", line 103, in <module>
    elif any(not r["conjecture_holds"] for r in results) and any(r["metric_value"] < 0.5 for r in results)
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b67d964.py", line 103, in <ge

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: Re-run the test with a larger number of instances and ensure that it completes successfully to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22002
2	preregistration	ollama_remote	glm4:latest	0	0	9273
3	novelty	ollama_remote	glm4:latest	0	0	12699
4	novelty	ollama_remote	glm4:latest	0	0	9144
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16527

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15297
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13190
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12893
9	judge	ollama_remote	glm4:latest	0	0	11724

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122749 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/0ada91edca54.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0ada91edca54.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0ada91edca54.tar.gz (if generated)